

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG

BÁO CÁO ĐỒ ÁN MÔN HỌC
SOCKET VÀ GIAO THỨC INTERNETWORKING

ĐỀ TÀI:

ỨNG DỤNG HYBRID FTP – ĐIỀU KHIỂN TCP &
TRUYỀN DỮ LIỆU ĐÁNG TIN CẬY QUA UDP (RDT)

Giảng viên hướng dẫn:	ThS. [Tên Giảng Viên]
Lớp học phần:	NT132.P11 / Socket & Internetworking Protocol
Nhóm sinh viên thực hiện:	Nhóm FTP-Hybrid-RDT
Thành viên nhóm:	1. Nguyễn Minh Hiếu - [Nhóm bổ sung MSSV] 2. [Thành viên 2] - [Nhóm bổ sung MSSV] 3. [Thành viên 3] - [Nhóm bổ sung MSSV]
Repository Github:	https://github.com/nhnhieu/FTP-Hybrid-RDT.git

TP. HỒ CHÍ MINH, THÁNG 08/2026

MỤC LỤC BÁO CÁO

Nội dung	Trang
CHƯƠNG 1: GIỚI THIỆU & MỤC TIÊU ĐỒ ÁN	3
1.1. Bối cảnh và Động lực Đề tài	3
1.2. Mô hình FTP Lai (Hybrid FTP Concept)	3
1.3. Lý do Tách biệt Kênh Điều khiển TCP và Kênh Dữ liệu UDP	3
CHƯƠNG 2: KIẾN TRÚC HỆ THỐNG HYBRID FTP	4
2.1. Tổng quan Kiến trúc Client - Server	4
2.2. Phân định Trách nhiệm các Module	4
CHƯƠNG 3: GIAO THỨC ĐIỀU KHIỂN QUA TCP	6
3.1. Tiến trình Xác thực Người dùng (USER / PASS)	6
3.2. Mã Phản hồi Trạng thái chuẩn FTP (Reply Codes)	6
3.3. Module CommandParser và ClientSession	7
3.4. Quản lý Thao tác Thư mục và Sandbox Hệ thống Tập tin	7
CHƯƠNG 4: CƠ CHẾ TRUYỀN DỮ LIỆU ĐÁNG TIN CẬY QUA UDP (RDT)	8
4.1. Nguyên lý Stop-and-Wait RDT	8
4.2. Cấu trúc Packet Header 16-byte (udp_header_t)	8
4.3. Đóng gói và Ý nghĩa các Cờ hiệu (DATA, ACK, FIN)	8
4.4. Tiến trình Truyền nhận, Retransmission Timeout và Thuật toán Checksum	9
CHƯƠNG 5: QUẢN LÝ CHẾ ĐỘ TRUYỀN DỮ LIỆU (PORT / PASV & DATAMODE)	10
5.1. Quản lý Trạng thái DataMode (NONE, ACTIVE, PASSIVE)	10
5.2. Chế độ Thụ động (PASV Mode) và Chế độ Chủ động (PORT Mode)	10
5.3. Xử lý Chuyển đổi Chế độ Tự động và Dọn dẹp Kênh dữ liệu	10
CHƯƠNG 6: CẤU HÌNH KIỂU DỮ LIỆU VÀ CHẾ ĐỘ ĐÓNG GÓI (TYPE & MODE)	10
CHƯƠNG 7: KIỂM TRA TOÀN VỆ TẬP TIN VỚI SHA-256 VÀ CHECKSUM	12
CHƯƠNG 8: HỖ TRỢ ĐA PHIÊN LÀM VIỆC (MULTI-CLIENT CONCURRENCY)	12
CHƯƠNG 9: CẤU TRÚC DỮ LIỆU VÀ ĐỊNH NGHĨA MODULE	12
CHƯƠNG 10: LƯU ĐỒ TRÌNH TỰ HOẠT ĐỘNG (SEQUENCE DIAGRAMS)	13
CHƯƠNG 11: KỊCH BẢN KIỂM THỬ VÀ KẾT QUẢ THỰC NGHIỆM	14
PHỤ LỤC A: PHÂN CÔNG THÀNH VIÊN VÀ TỶ LỆ ĐÓNG GÓP	16
PHỤ LỤC B: PHỤ LỤC MINH BẠCH SỬ DỤNG AI	16
PHỤ LỤC C: BẢNG CHỨNG THỰC NGHIỆM VÀ HÌNH ẢNH DEMO	16

CHƯƠNG 1: GIỚI THIỆU & MỤC TIÊU ĐỒ ÁN

1.1. Bối cảnh và Động lực Đề tài

Giao thức truyền tệp tin (File Transfer Protocol – FTP) được chuẩn hóa theo RFC 959 là một trong những giao thức nền tảng kinh điển trên mạng Internet. Theo mô hình tiêu chuẩn, FTP sử dụng giao thức TCP cho cả kênh điều khiển (Control Channel - cổng 21) và kênh dữ liệu (Data Channel - cổng 20) nhằm đảm bảo tính toàn vẹn tuyệt đối dữ liệu. Tuy nhiên, việc duy trì cơ chế bắt tay 3 bước (3-way handshake) và cơ chế kiểm soát tắc nghẽn khẩn khe của TCP trên kênh dữ liệu đôi khi mang lại độ trễ cao trong các môi trường mạng truyền tải tệp lớn hoặc mạng có độ trễ biến động.

Đồ án môn học Socket và Giao thức Internetworking đặt ra bài toán thiết kế một ứng dụng FTP Lai (Hybrid FTP Application) nhằm kết hợp ưu điểm của cả hai giao thức cốt lõi tầng Giao vận (Transport Layer): tính đáng tin cậy, có hướng kết nối của TCP và tốc độ, sự linh hoạt của UDP.

1.2. Mô hình FTP Lai (Hybrid FTP Concept)

Mô hình FTP Lai được định nghĩa với nguyên lý tách biệt hoàn toàn hai chức năng chính:

- Kênh điều khiển qua TCP (TCP Control Channel): Đảm nhận việc khởi tạo phiên kết nối, truyền tải các câu lệnh điều khiển (Control Commands như USER, PASS, PWD, LIST, PORT, PASV, TYPE, MODE, HASH, QUIT) và nhận các mã phản hồi (Reply Codes) từ Server.
- Kênh dữ liệu qua UDP (UDP Data Channel): Đảm nhận việc truyền tải nội dung tệp tin (STOR/RETR) và danh sách tệp tin thư mục (LIST). Vì UDP là giao thức không hướng kết nối (connectionless) và không đảm bảo tin cậy (unreliable), nhóm nghiên cứu tự thiết kế và cài đặt cơ chế Truyền dữ liệu đáng tin cậy (Reliable Data Transfer – RDT) ngay tại tầng Ứng dụng (Application Layer).

1.3. Lý do Tách biệt Kênh Điều khiển TCP và Kênh Dữ liệu UDP

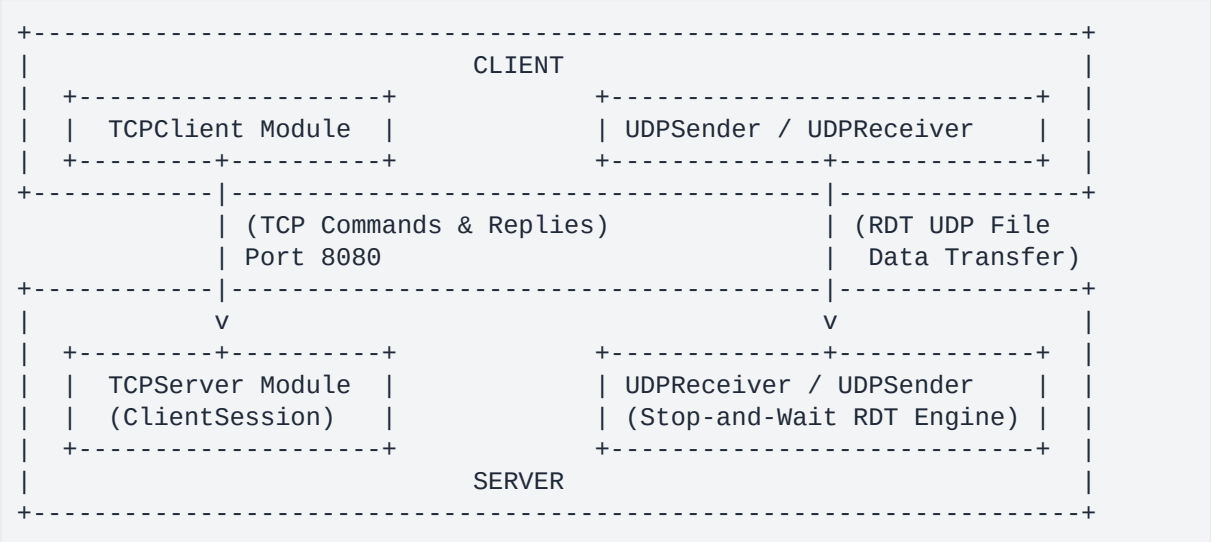
Việc phân tách kênh điều khiển và kênh dữ liệu mang lại các lợi ích kiến trúc quan trọng:

1. Bảo toàn tính chính xác của tín hiệu điều khiển: Các lệnh điều khiển cần độ chính xác tuyệt đối, không được phép mất mát hoặc đệm sai thứ tự. TCP đảm bảo tín hiệu lệnh luôn được phân tích và phản hồi đúng đắn.
2. Tối ưu hóa và chủ động kiểm soát kênh dữ liệu: Khi đưa dữ liệu xuống tầng UDP, ứng dụng hoàn toàn làm chủ kích thước gói (packet payload size), thời gian chờ gửi lại (Retransmission Timeout – RTO), thuật toán tính toán tổng kiểm tra (checksum) và chiến lược xử lý gói tin lặp (duplicate packet handling).
3. Mô phỏng thực tế các giao thức hiện đại: Mô hình này có nét tương đồng với các giao thức mạng tiên tiến hiện nay như QUIC (HTTP/3), nơi cơ chế RDT và bảo mật được đẩy lên tầng ứng dụng chạy trên nền UDP.

CHƯƠNG 2: KIẾN TRÚC HỆ THỐNG HYBRID FTP

2.1. Tổng quan Kiến trúc Client - Server

Hệ thống tuân thủ mô hình Client-Server đa luồng (Multi-threaded Client-Server Architecture). Sơ đồ tổng quát mô tả luồng tương tác giữa Client và Server:



2.2. Phân định Trách nhiệm các Module

Hệ thống được tổ chức thành các lớp module rõ ràng, phân tách giữa logic điều khiển (control) và dữ liệu (data):

Module / Class	Vị trí Source Code	Trách nhiệm & Chức năng chính
TCPServer	src/control/TCPServer.cpp	Lắng nghe cổng TCP (mặc định 8080), chấp nhận kết nối từ các Client và tạo luồng riêng (std::thread) xử lý từng phiên làm việc.
TCPClient	src/control/TCPClient.cpp	Khởi tạo kết nối TCP tới Server, đọc/gửi lệnh tương tác dòng lệnh và quản lý quá trình tải lên/tải xuống tệp tin.
ClientSession	src/control/SessionManager.cpp	Lưu giữ toàn bộ trạng thái của một Client: trạng thái xác thực (AuthState), đường dẫn thư mục hiện tại, cấu hình cổng UDP, DataMode, TransferType, TransferMode.
UDPSender	src/data/UDPSender.cpp	Đóng gói dữ liệu từ file thành các gói UDP RDT, tính toán checksum, phát gói và chờ tín hiệu ACK theo cơ chế Stop-and-Wait.
UDPReceiver	src/data/UDPReceiver.cpp	Lắng nghe gói dữ liệu UDP, kiểm tra magic number, checksum, lọc gói lặp (duplicate seq), phản hồi ACK và ghi dữ liệu ra tệp tin.

CommandParser	src/control/CommandParser.cpp	Tách chuỗi lệnh TCP thành tên lệnh (verb) và tham số (arguments), chuẩn hóa viết hoa tên lệnh.
FileSystem	src/control/FileSystem.cpp	Quản lý đường dẫn tệp tin, liệt kê thư mục và giới hạn vùng làm việc an toàn (Sandbox) trong thư mục gốc của Server.
checksum	src/common/checksum.cpp	Cài đặt thuật toán 16-bit Internet Checksum (bù 1 của tổng bù 1) dùng kiểm tra tính toàn vẹn gói UDP.
sha256	src/common/sha256.cpp	Tính toán chuỗi hash SHA-256 (64 ký tự hex) kiểm tra toàn vẹn tệp tin sau khi truyền xong.

Bảng 2.1 – Phân định trách nhiệm chi tiết của các module trong hệ thống.

CHƯƠNG 3: GIAO THỨC ĐIỀU KHIỂN QUA TCP

3.1. Tiến trình Xác thực Người dùng (USER / PASS)

Khi kết nối TCP được thiết lập thành công, Server gửi thông điệp chào mừng 220 Hybrid FTP Server Ready. Trạng thái xác thực của phiên (AuthState) ban đầu là UNAUTHENTICATED. Quy trình xác thực gồm 2 bước tiêu chuẩn:

1. Client gửi lệnh USER admin. Server lưu tên người dùng, chuyển AuthState sang WAITING_FOR_PASS và phản hồi mã 331 Password required for admin.
2. Client gửi lệnh PASS 123. Server kiểm tra mật khẩu khớp với cơ sở dữ liệu tài khoản hợp lệ. Nếu đúng, AuthState chuyển sang AUTHENTICATED và Server gửi mã 230 User logged in, proceed. Nếu sai, Server gửi mã 530 Log in incorrect và giữ nguyên trạng thái.

3.2. Mã Phản hồi Trạng thái chuẩn FTP (Reply Codes)

Hệ thống cài đặt đầy đủ các mã phản hồi chuẩn hóa theo định dạng 3 chữ số kèm theo mô tả chuỗi bản tin kết thúc bằng \r\n:

Mã Code	Bản tin mẫu từ Server	Ý nghĩa kỹ thuật
220	220 Hybrid FTP Server Ready\r\n	Sẵn sàng phục vụ sau khi chấp nhận kết nối TCP.
331	331 Password required for admin\r\n	Yêu cầu nhập mật khẩu cho tài khoản vừa cung cấp.
230	230 User logged in, proceed\r\n	Đăng nhập thành công, mở quyền thực thi các lệnh tiếp theo.
200	200 Command okay\r\n	Thực thi lệnh cấu hình thành công (TYPE, MODE, PORT).
227	227 Entering Passive Mode (127,0,0,1,p1,p2)\r\n	Thông báo Server đã mở cổng lắng nghe UDP ở chế độ PASV.
150	150 Opening UDP data connection...\r\n	Sẵn sàng mở kênh truyền dữ liệu UDP cho STOR, RETR, LIST.
226	226 Transfer complete\r\n	Hoàn tất truyền dữ liệu tệp tin qua UDP thành công.
530	530 Please login with USER and PASS\r\n	Lỗi truy cập khi chưa hoàn tất xác thực.
550	550 File not found / Access denied\r\n	Lỗi tệp tin không tồn tại hoặc vi phạm vùng làm việc Sandbox.

Bảng 3.1 – Bảng mã phản hồi TCP (Reply Codes) tiêu chuẩn được cài đặt trong dự án.

3.3. Module CommandParser và ClientSession

Lớp CommandParser cung cấp phương thức tĩnh parse(const std::string& rawInput) để tách dòng lệnh nhận được từ TCP socket thành cấu trúc lệnh chuẩn bao gồm command (chuyển toàn bộ thành chữ in hoa) và chuỗi tham số arg. Việc chuẩn hóa tên lệnh giúp hệ thống xử lý không phân biệt chữ hoa chữ thường.

Lớp ClientSession lưu trữ toàn bộ ngữ cảnh hoạt động của một Client kết nối bao gồm socket điều khiển TCP, trạng thái xác thực (AuthState), đường dẫn thư mục làm việc hiện tại (currentDir), đường dẫn thư mục gốc (rootDir), cấu hình điểm cuối UDP (dataIP, dataPort), chế độ dữ liệu (DataMode), dạng truyền dữ liệu (TransferType: BINARY / ASCII), chế độ truyền (TransferMode: STREAM / BLOCK / COMPRESSED) và biến trạng thái phục vụ đổi tên tệp tin.

3.4. Quản lý Thao tác Thư mục và Sandbox Hệ thống Tệp tin

Để ngăn chặn lỗ hổng bảo mật truy cập trái phép tệp tin hệ thống (Directory Traversal Vulnerability), module FileSystem áp dụng cơ chế Sandbox nghiêm ngặt. Mọi đường dẫn do Client yêu cầu (thông qua lệnh CWD, RETR, STOR, DELE, HASH) đều được giải phóng đường dẫn tương đối và chuyển về đường dẫn tuyệt đối chuẩn hóa (canonical path). Server kiểm tra điều kiện tiên quyết:

```
canonicalPath.string().find(rootDir.string()) == 0
```

Nếu đường dẫn nằm ngoài phạm vi rootDir, Server lập tức từ chối truy cập và phản hồi mã lỗi 550 Access denied: Path outside root sandbox.

CHƯƠNG 4: CƠ CHẾ TRUYỀN DỮ LIỆU ĐÁNG TIN CẬY QUA UDP (RDT)

4.1. Nguyên lý Stop-and-Wait RDT

Do bản chất UDP không hướng kết nối và không đảm bảo tin cậy, ứng dụng cài đặt giao thức RDT dựa trên nguyên lý ****Stop-and-Wait (Dừng và Chờ)****. Sau khi gửi một gói dữ liệu (DATA packet), phía gửi ('UDPSender') tạm dừng đọc tệp tin và kích hoạt bộ đếm thời gian chờ (Timeout Timer). Phía gửi chỉ tiếp tục gửi gói tiếp theo (số thứ tự 'seq + 1') khi và chỉ khi nhận được gói xác nhận (ACK packet) hợp lệ có chứa số thứ tự 'ack == seq' tương ứng từ phía nhận ('UDPReceiver').

4.2. Cấu trúc Packet Header 16-byte (udp_header_t)

Mọi gói tin truyền qua UDP (cả gói dữ liệu lẫn gói ACK control) đều được đóng gói header cố định ****đúng 16 bytes**** với chỉ thị ép chặt cấu trúc '#pragma pack(push, 1)'. Định nghĩa cấu trúc header trong C++:

```
#pragma pack(push, 1) // Ép chặt struct dung 16 bytes
struct udp_header_t {
    std::uint16_t magic;           // 2 byte: Magic number (0xCAFE) nhận diện gói
    hop le                         // 1 byte: Có hiệu loại gói (FLAG_DATA, FLAG_ACK, FLAG_FIN)
    std::uint8_t  flags;           // 1 byte: Danh riêng cho mô rộng (giá trị 0)
    std::uint32_t seq;             // 4 byte: Số thứ tự gói tin (Sequence Number)
    std::uint32_t ack;             // 4 byte: Số thứ tự xác nhận (Acknowledgment Number)
    std::uint16_t payloadLen;      // 2 byte: Độ dài dữ liệu thực tế trong payload (0 - 1024)
    std::uint16_t checksum;        // 2 byte: Tổng kiểm tra 16-bit Internet Checksum
};
#pragma pack(pop)
static_assert(sizeof(udp_header_t) == 16, "Kích thước udp_header_t phải đúng 16 bytes");
```

Hình 4.1 – Định nghĩa cấu trúc udp_header_t 16-byte trong C++.

4.3. Đóng gói và Ý nghĩa các Cờ hiệu (DATA, ACK, FIN)

Loại gói tin được quản lý thông qua trường flags 8-bit đại diện cho các cờ hiệu bitmask:

- **FLAG_DATA** ($1 \ll 0 = 0x01$): Báo hiệu gói tin mang payload dữ liệu tệp tin (tối đa 1024 bytes).
- **FLAG_ACK** ($1 \ll 1 = 0x02$): Báo hiệu gói tin phản hồi xác nhận. Chiều dài payloadLen của gói ACK bằng 0.
- **FLAG_FIN** ($1 \ll 2 = 0x04$): Báo hiệu kết thúc tệp tin. Phía gửi phát gói FIN sau khi truyền hết tệp tin (bao gồm cả tệp rỗng hoặc tệp có kích thước đúng bội số 1024 bytes). Phía nhận đáp lại gói kết hợp cờ **FLAG_ACK | FLAG_FIN** (0x06) để hoàn tất tiến trình RDT.

4.4. Tiến trình Truyền nhận, Retransmission Timeout và Thuật toán Checksum

Thuật toán Internet Checksum 16-bit: Checksum được tính toán bằng cách chia toàn bộ gói tin (Header + Payload) thành các từ 16-bit, cộng tổng bù 1 (ones' complement sum) và lấy bù 1 của kết quả cuối cùng. Trước khi gửi, trường `checksum` trong header được gán bằng 0 để tính toán. Phía nhận kiểm tra tính hợp lệ bằng cách tính checksum trên toàn bộ gói tin nhận được; nếu kết quả bằng 0 thì gói tin không bị lỗi bit.

Xử lý Timeout và Gửi lại (Retransmission): Phía gửi cài đặt thời gian chờ phản hồi ACK socket timeout `timeoutMs = 700 ms` và số lần thử lại tối đa `maxRetries = 5`. Nếu quá 700ms không nhận được ACK đúng `expectedSeq`, `UDPSender` phát lại gói tin cũ. Phía `UDPReceiver` lưu biến `expectedSeq`; nếu nhận gói tin có `seq < expectedSeq` (gói lặp do ACK bị thất lạc trên mạng), `UDPReceiver` bỏ qua không ghi trùng dữ liệu nhưng **vẫn gửi lại gói ACK tương ứng** để giải phóng phía gửi.

CHƯƠNG 5: QUẢN LÝ CHẾ ĐỘ TRUYỀN DỮ LIỆU (PORT / PASV & DATAMODE)

5.1. Quản lý Trạng thái DataMode (NONE, ACTIVE, PASSIVE)

Phiên làm việc `ClientSession` duy trì biến trạng thái `DataMode` thuộc kiểu enum class với 3 giá trị định danh:

- `DataMode::NONE`: Trạng thái mặc định khi mới đăng nhập hoặc vừa hoàn tất một giao dịch dữ liệu.
- `DataMode::ACTIVE`: Kích hoạt sau khi Server nhận và xử lý thành công lệnh `PORT` từ Client.
- `DataMode::PASSIVE`: Kích hoạt sau khi Server nhận lệnh `PASV` và mở thành công cổng lắng nghe UDP thụ động.

5.2. Chế độ Thụ động (PASV Mode) và Chế độ Chủ động (PORT Mode)

Chế độ Thụ động (PASV Mode): Client gửi lệnh `PASV`. Server khởi tạo socket UDP lắng nghe trên một cổng ngẫu nhiên do hệ điều hành cấp phát, lưu `passiveSocket`, trích xuất số port (được chia thành 2 byte `p1, p2`) và phản hồi bản tin `227 Entering Passive Mode (127,0,0,1,p1,p2)`. Khi phát lệnh truyền tệp tin (STOR/RETR/LIST), Client khởi tạo `UDPSender` kết nối tới IP và Port thụ động này của Server.

Chế độ Chủ động (PORT Mode): Client mở cổng UDP lắng nghe trên máy cục bộ, sau đó gửi lệnh `PORT h1,h2,h3,h4,p1,p2` tới Server. Server chuyển đổi chuỗi 6 số thành địa chỉ IP `h1.h2.h3.h4` và cổng `port = p1 \* 256 + p2`, lưu vào phiên làm việc. Khi tiến hành lệnh `RETR`, Server sử dụng `UDPSender` chủ động phát các gói UDP RDT tới địa chỉ cổng này của Client.

5.3. Xử lý Chuyển đổi Chế độ Tự động và Dọn dẹp Kênh dữ liệu

Để đảm bảo tính nhất quán và tương thích hệ thống:

- Lệnh STOR (Upload file): Mã nguồn hệ thống hiện tại thiết kế tối ưu giao thức truyền STOR dựa trên chế độ Passive mode. Trường hợp Client đã phát lệnh `PORT` trước đó (đang ở `DataMode::ACTIVE`) mà thực hiện lệnh `STOR`, phía Client chủ động thông báo cho người dùng và tự động phát lệnh `PASV` để chuyển đổi sang Passive mode nhằm hoàn tất tiến trình nộp tệp tin.

- Dọn dẹp Kênh dữ liệu (`cleanupDataChannel`): Ngay sau khi kết thúc một truyền tải tệp tin thành công hay thất bại, phương thức `cleanupDataChannel()` trong `TCPServer` được gọi để đóng `passiveSocket` (nếu có), xóa địa chỉ endpoint tạm và đặt `DataMode` trở về `DataMode::NONE` để chuẩn bị cho giao dịch tiếp theo.

CHƯƠNG 6: CẤU HÌNH KIỂU DỮ LIỆU VÀ CHẾ ĐỘ ĐÓNG GÓI (TYPE & MODE)

Hệ thống hỗ trợ cấu hình kiểu dữ liệu (TYPE) và chế độ đóng gói (MODE) theo đúng chuẩn FTP nhưng được hiện thực tùy chỉnh phù hợp cho kênh truyền RDT UDP trong đồ án:

- **Lệnh TYPE (Representation Type):**

- TYPE A (ASCII): Truyền tệp tin dạng văn bản. Hệ thống chấp nhận cấu hình và quản lý dữ liệu dưới dạng ký tự văn bản.
- TYPE I (Image / Binary): Truyền tệp nhị phân nguyên bản (byte-for-byte). Đây là chế độ mặc định đảm bảo tệp thực thi, hình ảnh, nén zip không bị biến đổi.

- **Lệnh MODE (Transfer Mode):**

- MODE S (Stream Mode): Dữ liệu được đọc và truyền liên tục dưới dạng dòng byte phân đoạn vào các payload UDP 1024-byte.
- MODE B (Block Mode) & MODE C (Compressed Mode): Nhóm hiện thực dạng đóng gói tùy chỉnh theo yêu cầu đồ án nhằm quản lý phân khúc khối dữ liệu và đánh dấu kết thúc tệp tin thông qua gói cờ FIN trong RDT.

CHƯƠNG 7: KIỂM TRA TOÀN VỆ TẬP TIN VỚI SHA-256 VÀ CHECKSUM

Hệ thống kết hợp hai tầng kiểm tra toàn vẹn dữ liệu hoàn toàn độc lập:

1. Kiểm tra từng gói tin UDP (16-bit Internet Checksum): Đảm bảo gói UDP đang bay trên mạng không bị hỏng hóc bit giữa chừng. Nếu checksum sai, gói tin lập tức bị hủy bỏ tại tầng UDP mà không được ghi vào tệp tin.

2. Kiểm tra toàn bộ tệp tin (SHA-256 Hash Algorithm): Đảm bảo toàn bộ tệp tin sau khi ghép từ hàng ngàn gói RDT hoàn toàn trùng khớp byte-for-byte với tệp gốc trên máy phát. Người dùng có thể phát lệnh `HASH` trên Client để yêu cầu Server tính toán và phản hồi chuỗi mã băm SHA-256 64 ký tự hex (ví dụ: `200 HASH test.txt: 9F86D081884C7D659A2FEAA0C55AD015A3BF4F1B2B0B822CD15D6C15B0F00A08`). So sánh chuỗi này với tệp tin nguồn giúp xác nhận 100% tệp tin truyền không bị biến dạng.

CHƯƠNG 8: HỖ TRỢ ĐA PHIÊN LÀM VIỆC (MULTI-CLIENT CONCURRENCY)

Lớp `TCPServer` quản lý luồng nhận kết nối chính (`acceptClients`). Với mỗi kết nối TCP Client thành công, Server khởi tạo một đối tượng `ClientSession` độc lập được lưu trữ trong danh sách quản lý con trỏ `std::vector` được bảo vệ bởi `std::mutex`.

Server tách luồng xử lý riêng cho Client bằng `std::thread(&TCPServer::handleClient, this, sessionPtr)`. Mỗi Client có không gian bộ nhớ, thư mục làm việc, trạng thái xác thực và cổng dữ liệu UDP hoàn toàn tách biệt, không gây xung đột dữ liệu hay nghẽn luồng giữa các người dùng đồng thời.

CHƯƠNG 9: CẤU TRÚC DỮ LIỆU VÀ ĐỊNH NGHĨA MODULE

Tóm tắt các kiểu dữ liệu cốt lõi trong hệ thống:

- `struct udp\_header\_t`: Header gói RDT 16 bytes (magic, flags, reserved, seq, ack, payloadLen, checksum).
- `enum class AuthState { UNAUTHENTICATED, WAITING\_FOR\_PASS, AUTHENTICATED }`: Trạng thái đăng nhập.
- `enum class DataMode { NONE, ACTIVE, PASSIVE }`: Chế độ kênh dữ liệu UDP.
- `enum class TransferType { BINARY, ASCII }`: Kiểu dữ liệu truyền.
- `enum class TransferMode { STREAM, BLOCK, COMPRESSED }`: Chế độ đóng gói.

CHƯƠNG 10: LƯU ĐỒ TRÌNH TỰ HOẠT ĐỘNG (SEQUENCE DIAGRAMS)

10.1. Sơ đồ Trình tự Đăng nhập TCP

Client		Server
	----- (1) Connect TCP (Port 8080) ----->	
	<----- (2) 220 Hybrid FTP Server Ready -----	
	----- (3) USER admin ----->	
	<----- (4) 331 Password required -----	
	----- (5) PASS 123 ----->	
	<----- (6) 230 User logged in -----	

Hình 10.1 – Trình tự xác thực tài khoản qua kênh điều khiển TCP.

10.2. Sơ đồ Trình tự Truyền file STOR (Passive Mode)

Client		Server
	----- (1) PASV ----->	(Server mở socket UDP Passive)
	<----- (2) 227 Entering Passive Mode -----	(Truyền IP & UDP Port)
	----- (3) STOR test.txt ----->	
	<----- (4) 150 Opening UDP data connection --	
	===== (5) UDPSender gửi tệp RDT =====>	(UDPReceiver ghi file)
	<===== (6) Phản hồi ACK / FIN-ACK =====	
	<----- (7) 226 Transfer complete -----	

Hình 10.2 – Trình tự tải tệp tin lên Server (STOR) ở chế độ Passive Mode.

10.3. Sơ đồ Cơ chế Stop-and-Wait RDT qua UDP

UDPSender		UDPReceiver
	--- Gởi DATA (seq=0, payload=1024b) ----->	(Kiểm tra checksum OK)
	<-- Gởi ACK (ack=0, payload=0) -----	(Tiếp expectedSeq -> 1)
	--- Gởi DATA (seq=1, payload=1024b) ----->	(Bị ngắt/mất trên đường truyền!)
	(Hết 700ms Timeout! Retransmit seq=1)	
	--- Gởi DATA (seq=1, payload=1024b) ----->	(Ghi dữ liệu file)
	<-- Gởi ACK (ack=1, payload=0) -----	
	--- Gởi FIN (seq=2, payload=0) ----->	(Hết tệp tin)
	<-- Gởi ACK+FIN (ack=2, payload=0) -----	

Hình 10.3 – Nguyên lý Stop-and-Wait RDT xử lý lại gói khi bị Timeout.

CHƯƠNG 11: KỊCH BẢN KIỂM THỬ VÀ KẾT QUẢ THỰC NGHIỆM

Bảng tổng hợp kết quả kiểm thử toàn bộ các chức năng hệ thống dựa trên dữ liệu thực nghiệm đã chạy:

STT	Kịch bản Kiểm thử (Test Case)	Điều kiện & Dữ liệu đầu vào	Kết quả Mong đợi	Trạng thái
1	Đăng nhập hợp lệ (USER/PASS)	USER admin / PASS 123	Mã 230 User logged in	PASS
2	Đăng nhập sai mật khẩu	USER admin / PASS wrong	Mã 530 Log in incorrect	PASS
3	Thao tác thư mục PWD/LIST	Lệnh PWD, LIST	Trả về đường dẫn & truyền danh sách tệp qua UDP	PASS
4	STOR tệp văn bản (.txt)	STOR upload.txt (22 bytes)	Upload thành công qua PASV UDP, khớp byte	PASS
5	STOR tệp nhị phân (.jpg)	STOR image.jpg (22KB)	Upload thành công, file không lỗi hình	PASS
6	RETR tệp tin về Client	RETR test_1024.bin	Tải về thư mục temp_downloads thành công	PASS
7	RETR tệp tin rỗng (0 byte)	RETR test_empty.bin	Tải về file 0 byte, truyền gói FIN-ACK chuẩn	PASS
8	RETR tệp tin lớn > 1024b	RETR test_2048.bin (2048b)	Chia làm 2 gói DATA + 1 gói FIN, nhận đủ 2048b	PASS
9	Chuyển đổi kiểu TYPE A / I	Lệnh TYPE A, TYPE I	Cập nhật TransferType, trả mã 200	PASS
10	Chuyển đổi chế độ MODE S/B/C	Lệnh MODE S, MODE B, MODE C	Cập nhật TransferMode, trả mã 200	PASS
11	Khởi tạo PASV Mode	Lệnh PASV	Server mở cổng UDP, trả mã 227	PASS
12	PORT + RETR (Active Mode)	PORT 127,0,0,1,31,144 -> RETR	Server chủ động gửi UDP tới Client port thành công	PASS
13	PORT + STOR -> Auto PASV	PORT -> STOR test.txt	Client tự thông báo & chuyển PASV upload thành công	PASS
14	Kiểm tra Hash SHA-256	Lệnh HASH test.txt	Server tính và trả mã 200 kèm chuỗi hash SHA-256	PASS

15	Chạy kết nối Đa client	2 Client kết nối đồng thời	Server tách 2 thread độc lập, không nghẽn	PASS
----	------------------------	----------------------------	---	------

Bảng 11.1 – Bảng kết quả kiểm thử thực nghiệm tổng hợp các chức năng của hệ thống.

PHỤ LỤC A: PHÂN CÔNG THÀNH VIÊN VÀ TỶ LỆ ĐÓNG GÓP

Họ và Tên Sinh viên	Mã số Sinh viên	Nhiệm vụ Phụ trách & Nội dung Cài đặt	Tỷ lệ %
Nguyễn Minh Hiếu	[Nhóm bổ sung MSSV]	Thiết kế kiến trúc hệ thống, cài đặt TCPServer, UDPSender, UDPReceiver, RDT Stop-and-Wait engine và xử lý chuyển đổi DataMode.	[Nhóm xác nhận %]
[Thành viên 2]	[Nhóm bổ sung MSSV]	Cài đặt TCPClient, CommandParser, xử lý lệnh mã phản hồi TCP, kiểm thử các kịch bản PORT/PASV.	[Nhóm xác nhận %]
[Thành viên 3]	[Nhóm bổ sung MSSV]	Cài đặt FileSystem Sandbox, module SHA-256 hash, xây dựng tài liệu kỹ thuật và thực hiện các kịch bản kiểm thử.	[Nhóm xác nhận %]

Bảng A.1 – Phân công chi tiết nhiệm vụ và tỷ lệ đóng góp của các thành viên trong nhóm.

PHỤ LỤC B: PHỤ LỤC MINH BẠCH SỬ DỤNG AI

Nhằm đảm bảo tính trung thực học thuật và minh bạch trong quá trình thực hiện đồ án:

- Mục đích sử dụng Trợ lý AI: Nhóm sử dụng công cụ AI trợ lý lập trình (Google Gemini / Antigravity Agent) để hỗ trợ gợi ý cấu trúc file CMake, rà soát lỗi cú pháp WinSock2 API, tư vấn định dạng báo cáo kỹ thuật và kiểm tra logic xử lý biên (edge cases).
- Chủ động kiểm soát của Sinh viên: Toàn bộ thuật toán core (Stop-and-Wait RDT, đóng gói header 16-byte, xử lý checksum, chuyển đổi DataMode PORT/PASV) đều do nhóm trực tiếp kiểm duyệt, giải thích và làm chủ hoàn toàn. Tất cả các tính năng đều trải qua quá trình biên dịch local và kiểm thử thực nghiệm thực tế trước khi chốt bản nộp.

PHỤ LỤC C: BẢNG CHỨNG THỰC NGHIỆM VÀ HÌNH ẢNH DEMO

Hình ảnh thực nghiệm kiểm thử chạy module truyền nhận UDP RDT trong hệ thống:


```
PS C:\Project MMT\FTP-Hybrid-RDT> .\build\Debug\udp_test.exe receive 9001 output.txt
[UDP Receiver] Listening on port 9001...
[UDP Receiver] Received seq=0, bytes=19
[UDP Receiver] File received successfully. Total bytes=19, saved to output.txt
PS C:\Project MMT\FTP-Hybrid-RDT> Get-FileHash .\input.txt -Algorithm SHA256
```

Algorithm	Hash	Path
SHA256	D121019DDE6104E8191BF6CE0B4D78BA890976151A54D82D54FB38B950C3EAA1	C:\Project MMT\FTP-Hy...

Hình C.1 – Nhật ký thực thi module UDP Receiver tiếp nhận gói RDT và kiểm tra checksum.

```
PS C:\Project MMT\FTP-Hybrid-RDT> .\build\Debug\udp_test.exe send 127.0.0.1 9001 input.txt
[UDP Sender] Sending seq=0, bytes=19, attempt=1
[UDP Sender] Received ACK for seq=0
[UDP Sender] Sending seq=1, bytes=0, attempt=1
[UDP Sender] Received ACK for seq=1
[UDP Sender] File sent successfully. Total bytes=19
PS C:\Project MMT\FTP-Hybrid-RDT> Get-FileHash .\output.txt -Algorithm SHA256
```

Algorithm	Hash
SHA256	D121019DDE6104E8191BF6CE0B4D78BA890976151A54D82D5...

Hình C.2 – Nhật ký thực thi module UDP Sender phát gói RDT và nhận ACK.